GUIA VISUAL 073

ARQUIVOS, DESCRITORES E PROCESSOS EM SCRIPTS

*O analista queria só deixar a lista de alvos ordenada no lugar: `sort alvos.txt > alvos.txt`. O arquivo ficou com **zero bytes** — sem mensagem de erro, sem confirmação, e o backup mais recente era de três dias antes. A explicação não está no `sort`: está na **ordem em que o shell faz as coisas**. Ele prepara os redirecionamentos **antes** de executar o comando, e `> arquivo` quer dizer "abra truncando". Quando o `sort` foi ler, o arquivo já estava vazio havia milissegundos.*

073
GUIAS LINUX

A TABELA DE DESCRITORES · E A ORDEM QUE ESVAZIOU O ARQUIVO

SEU SCRIPT
bash backup.sh
PID 4821

0	stdin	→	/dev/pts/1 o terminal
1	stdout	→	saida.txt exec 1> saida.txt
2	stderr	→	/dev/pts/1 o terminal
3	livre	→	dados.csv exec 3< dados.csv
9	livre	→	rastro.log BASH_XTRACEFD=9
200	livre	→	meu.lock flock -n 200

AS FORMAS DE REDIRECIONAR

`> arq`	abre para escrita, **truncando**
`>> arq`	escreve acrescentando ao fim
`< arq`	abre para leitura
`<> arq`	leitura **e** escrita, sem truncar
`>| arq`	escreve mesmo com `noclobber` ligado
`2>&1`	o 2 passa a apontar para onde o 1 aponta **naquele momento**
`&> arq`	atalho do Bash para `> arq 2>&1`
`n> arq`	redireciona o descritor n; `n>&m` duplica; `n>&-` fecha

`sort a.txt > a.txt`

1	2	3	4
shell prepara o redirecionamento	a.txt **truncado**	sort só agora começa a rodar	lê 0 bytes, grava 0 bytes — **sem aviso**

`sort a > tmp && mv tmp a`

1	2	3	4
mktemp **no mesmo diretório**	sort lê o original, grava no temporário	o **&&** só segue se deu certo	mv renomeia **atomicamente**

A regra vale para qualquer transformação de arquivo, não só o `sort`: leia de um lugar, escreva em outro, troque no fim. O ponto de atomicidade é o `mv` **dentro do mesmo sistema de arquivos** — entre sistemas diferentes ele vira cópia mais remoção, e o intervalo entre as duas é exatamente a janela que se queria eliminar. Por isso o temporário nasce ao lado do original, e não em `/tmp`. Se o `sort` falhar, o `&&` impede o `mv` e o original fica intacto. Alternativas: `sponge` (do `moreutils`) absorve toda a entrada antes de abrir o destino — `sort alvos.txt | sponge alvos.txt` — ao custo de manter tudo em memória; e `sed -i` / `gawk -i inplace` resolvem, mas **não são atômicos de verdade**: gravam num temporário e renomeiam, o que troca o inode e **quebra hard links**. Como rede extra, `set -o noclobber` transforma sobrescrita acidental em erro, e `>|` é a permissão explícita. E a pegadinha da ordem: `cmd > log 2>&1` manda os dois para o arquivo, mas `cmd 2>&1 > log` manda o **stderr para o terminal** — quando o `2>&1` é processado, o 1 ainda aponta para lá. Leia da esquerda para a direita, como uma sequência de atribuições.

LER LINHA A LINHA · EXEC · FLOCK

kali@kali: ~/lab-descritores

```
# o laço canônico — cada peça corrige um defeito
$ while IFS= read -r linha || [[ -n "$linha" ]]; do
    printf '[%s]\n' "$linha"
  done < entrada.txt
# comando de dentro que lê stdin engole o resto: use outro fd
$ while IFS= read -r -u 3 host; do ssh "$host" uptime; done 3< hosts.txt
# carregar tudo num array de uma vez
$ mapfile -t alvos < <(grep -v '^#' escopo.txt)
$ printf 'total: %d\n' "${#alvos[@]}"
total: 42
# registrar TUDO sem perder a tela — uma linha no topo do script
$ exec > >(tee -a /var/log/meuscript.log) 2>&1
# salvar e restaurar o stdout
$ exec 3>&1; exec 1> /tmp/relatorio.txt; echo linha; exec 1>&3 3>&-
# trava: nada de arquivo .pid — o kernel solta até em kill -9
$ exec 200> /var/lock/meu.lock
$ flock -n 200 || { echo "ja ha uma execucao" >&2; exit 1; }
# dois consumidores da mesma saída, e um teste de porta sem nc
$ gerar | tee >(gzip > r.gz) >(sha256sum > r.sha) > r.txt
$ timeout 2 bash -c 'exec 3<>/dev/tcp/127.0.0.1/22' && echo "22 aberta"
22 aberta
```

Higiene do temporário, em quatro linhas: `umask 077` **antes** de criar; respeite o `TMPDIR` (em serviço systemd com `PrivateTmp=yes` ele aponta para um diretório isolado); `TRABALHO="$(mktemp -d -t meuscript.XXXXXXXX)"`, com molde nomeado para o diretório ser identificável no `ls /tmp` quando algo der errado; e `trap 'rm -rf -- "$TRABALHO"' EXIT`. Para segredo que não deve tocar o disco, `mktemp -p /dev/shm` — memória, sem rastro em disco, mas **ainda legível por quem tem privilégio** na máquina.

O LAÇO CANÔNICO, PEÇA POR PEÇA

`IFS=`	vazio, impede que **espaços e tabulações das bordas** sejam removidos
`-r`	não interpreta a barra invertida — sem ele, `C:\novo` vira `C:ovo`
`|| [[ -n ]]`	salva a **última linha sem quebra final**: sem isso ela é lida mas o `read` devolve 1 e o laço a descarta em silêncio
`< arq no done`	e nunca `cat arq |`: o pipe cria subshell e **toda variável modificada no laço se perde**. Para saída de comando, `done < <(cmd)`

EXEC E FLOCK

`exec`	sem comando, aplica o redirecionamento ao **shell inteiro** — todos os comandos seguintes herdam
`flock -n`	falha na hora em vez de esperar; sem `-n`, a segunda cópia fica na fila. **Não precisa apagar o lock:** o kernel libera até em `SIGKILL` — é a vantagem sobre o arquivo `.pid`, que deixa órfão em toda morte anormal
`<( ... )`	vira **caminho de arquivo** (`/dev/fd/63`), o que alimenta programa que não aceita pipe: `diff <(sort a) <(sort b)`
`>( ... )`	o shell **não espera** o processo de dentro terminar — ler o arquivo na linha seguinte pode pegá-lo incompleto. Use `wait`
`/dev/tcp`	socket implementado **pelo próprio Bash**: não existe no sistema de arquivos e não funciona em **dash** nem **sh**

O SCRIPT COMO PROCESSO

`$$`	PID do shell atual — **não muda** em subshell
`BASHPID`	PID real, que **muda** em subshell
`$! · PPID`	último posto em segundo plano · o pai
`BASH_SOURCE`	`[[ "${BASH_SOURCE[0]}" == "$0" ]]` distingue **executado** de **carregado com source** — é o que deixa o mesmo arquivo ser biblioteca e programa

Todo redirecionamento com `>` é uma destruição silenciosa em potencial. Não há confirmação, não há mensagem, e o conteúdo anterior não é recuperável: `sort a.txt > a.txt`, `cmd > $VAZIA` e um `>` digitado onde se queria `>>` apagam arquivo de escopo, de resultado ou de log sem deixar rastro. As três defesas, em ordem: **(1)** nunca escreva no arquivo que está lendo — `tmp="$(mktemp "${arq}.XXXXXX")"; cmd "$arq" > "$tmp" && mv -- "$tmp" "$arq"`; **(2)** `set -o noclobber` no topo do script, que transforma sobrescrita em erro e exige `>|` para o caso intencional; **(3)** `set -u`, para que `> "$DESTINO"` com a variável vazia falhe em vez de virar erro de sintaxe ou arquivo perdido. Ensaie qualquer script novo em cópia descartável do diretório, nunca no artefato original.

RESUMO RÁPIDO

- ✓ **O shell prepara os redirecionamentos antes de executar o comando** — por isso `sort a > a` esvazia o arquivo. Leia de um lugar, escreva em outro, troque com `mv` no fim.
- ✓ **Redirecionar é apontar um descritor para outro destino.** `2>&1` copia o destino **atual** do 1: `cmd > log 2>&1` e `cmd 2>&1 > log` fazem coisas diferentes.
- ✓ **`exec` sem comando redireciona o shell inteiro**: `exec 3>&1` salva a saída; `exec > >(tee -a log) 2>&1` registra tudo sem perder a tela.
- ✓ **O laço canônico é `while IFS= read -r linha || [[ -n "$linha" ]]; do … done < arquivo`** — cada peça corrige um defeito: bordas, barra invertida, última linha e o subshell do pipe.
- ✓ **Comando dentro do laço que lê `stdin` engole o resto do arquivo.** Use `-u 3` com `3< arquivo`, ou `< /dev/null` no comando de dentro.
- ✓ **`mktemp` no mesmo diretório do original** é o que torna o `mv` final atômico; entre sistemas de arquivos diferentes, deixa de ser.
- ✓ **`flock` sobre um descritor é a trava correta**: o kernel a libera mesmo em `SIGKILL`, ao contrário do arquivo `.pid` criado à mão.
- ✓ **`<(…)` e `>(…)` viram caminhos de arquivo**, alimentando programa que não aceita pipe — lembrando que o shell não espera o processo de `>(…)`.
- ✓ **`umask 077` antes de `mktemp`, `/dev/shm` para segredo** e `trap … EXIT` para a remoção. Nome previsível em diretório público é vulnerabilidade, não descuido.

NO KALI

Coleta que grava enquanto lê é o desastre da abertura em escala. Um script que refina a lista de alvos a cada rodada e a grava por cima do próprio arquivo **perde o escopo inteiro** na primeira falha — e a lista de escopo é o artefato mais importante de um teste autorizado. Reconstruí-la de memória é como escopo vira excesso, com o enquadramento da Lei 12.737/2012. Temporário no mesmo diretório e `mv` no fim. **`flock` em varredura agendada não é preciosismo:** uma varredura que demora mais que o intervalo do agendamento acaba com **duas cópias disparando pacotes ao mesmo tempo contra o mesmo alvo** — o dobro da taxa combinada no contrato, sem que ninguém tenha pedido. `exec 200> /var/lock/varredura.lock; flock -n 200 || exit 0` é uma linha e resolve. **`exec > >(tee -a registro.log) 2>&1` no topo do script de coleta** dá registro completo do que foi executado com a saída ainda visível: num teste autorizado esse arquivo é a prova de que a atividade ficou dentro do combinado — sem ele, a defesa é a sua memória. **`/dev/shm` para credencial, `/tmp` nunca:** hash, token e chave temporária pertencem à memória, com `umask 077` antes e remoção garantida por `trap` — e em máquina compartilhada o `/dev/shm` reduz o rastro em disco, não elimina a exposição. **`/dev/tcp` verifica porta sem instalar nada** numa máquina onde você tem acesso legítimo mas não pode instalar pacotes; é recurso de diagnóstico, e usá-lo para varrer faixa fora de escopo autorizado é a mesma infração que fazê-lo com `nmap`. **Família RHEL:** serviços com `PrivateTmp=yes` ganham um `/tmp` privado — um script que grava lá esperando que outro processo leia simplesmente não se encontra, e o `TMPDIR` é o caminho de coordenação; e o SELinux rotula arquivos, então criar lock à mão em `/var/lock` pode dar "Permission denied" com permissões POSIX corretas — o idiomático é `/run/lock` com `RuntimeDirectory=` na unidade.